TP2 - Consigna 3

Motivación:

Esta actividad tiene la finalidad de definir clases con relaciones entre ellas, de modo de resolver un requerimiento sencillo e implementarla en lenguaje C++, con la restricción de integrar lo nuevo con clases preexistentes, propias y/o de terceros.

Un aspecto a considerar en el desarrollo de software orientado a objetos, es el uso (habitual) de clases auxiliares.

Entre ellas, las clases destinada a registrar y/o administrar mensajes del sistema, errores, advertencias, información de depuración, etc.

En general, esta clase se denomina Logger (o registrador) y es usada como herramienta base para tareas de depuración y/o de optimización.

Una característica de la clase es la de definir diferentes niveles de registro, en función del tipo de mensaje y/o evento que se registra. Por ejemplo:

- DEBUG – Para mensajes de depuración detallada.
- INFO – Para mensajes informativos generales.
- WARNING – Para advertencias sobre situaciones inesperadas pero no críticas.
- ERROR – Para errores graves que no detienen la aplicación.
- CRITICAL – Para errores críticos que pueden detener el sistema.

Otro aspecto del desarrollo a considerar, es el de aprovechar las referencias al conocimiento ya adquirido y las buenas prácticas ya probadas. Algo de esto se encuentra detallado en los patrones de software.

Un patrón (pattern) es una solución ya probada y aplicable a un problema que se presenta una y otra vez en el desarrollo de distintas aplicaciones y en distintos contextos. Es importante destacar que un patrón no es en general una solución en forma de código directamente "listo para usar", sino más bien una descripción de cómo resolver el problema y de ante qué circunstancias es aplicable. Podría decirse que, un patrón es una regla de tres partes, la cual expresa una relación entre un cierto contexto, un problema y una solución.

Los patrones suelen agruparse en colecciones, según diferentes criterios (según la etapa o ámbito del desarrollo, según su finalidad, según el autor u organización inicial).

Una de las categorías son los patrones de diseño. Estos, ayudan a diseñar soluciones de software, definiendo comportamientos comunes que suelen encontrarse en distintos componentes de software

Particularmente, el patrón Singleton es un patrón de diseño que restringe la instanciación de una clase a un único objeto. Esto es útil cuando se necesita exactamente un objeto para coordinar las acciones de todo el sistema.

Requerimientos:

Se necesita integrar una clase auxiliar, que permita realizar un registro de los eventos que ocurran durante la ejecución de programas, almacenando los datos en un archivo de texto.

Para ello, otros integrantes del equipo de desarrollo han provisto de una clase Logger.

Para el sistema en curso, se desea reutilizar el código ya desarrollado para el primer requerimiento (clase File_NNN).

Para el sistema futuro, se sabe que, además de los eventos internos, para los cuales se requiere conocer en qué modulo se produjo (y eventualmente también en qué línea de programa) deberá registrar eventos que surgen de peticiones externas, para los cuales se requiere conocer un ID de usuario y un ID de dispositivo/aplicación remoto desde donde se solicitan servicios.

Esta clase auxiliar debe ser capaz también de ofrecer la información almacenada en formato CSV, JSON o XML según se necesite, ya sea de manera completa, por rango de fechas en que ocurrieron los eventos, y/o por tipo de evento y/o por usuario.

Para el requerimiento actual, usar un módulo principal capaz de construir los objetos que se requieran y realizar una secuencia de prueba significativa.

La implementación Orientada a Objetos realizada debe separar la capa de presentación/control de la de modelo.

Es conveniente que la clase implementada utilice excepciones para gestionar los errores y/o devolver mensajes destinados al usuario.

Procedimiento general:

1) Diseñar en Umbrello (o similar) un modelo OO usando UML, que muestre y defina la/s clase/s fundamentales. Realice las mismas consideraciones que las utilizadas en consignas previas.

2) Dado que se debe aprovechar el modelo y las definiciones para la clase archivo desarrollada anteriormente, analice al menos 3 relaciones diferentes entre aquella y la clase Logger provista.

3) Generar los módulos que correspondan, con el código base en lenguaje C++, para uno de los modelos propuestos.

4) Editar y completar la implementación de modo de obtener una aplicación funcional que resuelva el requerimiento.

5) Realizar un informe similar a lo ya pautado, incluyendo sus comentarios al análisis de relaciones realizado y el modelo adoptado.

6) A partir del modelo implementado, analice y agregue al informe un detalle con los cambios que debería hacer en la clase para que cumpla con el patrón Singleton (la implementación de esta característica es opcional)

7) Preparar una carpeta para entrega, similar a lo ya pautado.

Recursos complementarios (disponibles en el aula virtual)**:**

- Apuntes de clase.
- Referencias de las consignas previas.
- Libro: Patrones de Diseño de Gamma, Helm, Johnson y Vlissides

Consigna 4

Motivación:

Esta actividad tiene la finalidad de integrar elementos de orientación a objetos propios y de terceros con comunicaciones en redes de datos, usando programas en lenguaje C++.

En la Orientación a Objetos, un ***paquete*** (package) es una forma de organizar y agrupar clases, interfaces, y otros elementos relacionados dentro de un sistema. Sirve para estructurar el código, facilitar el mantenimiento y controlar la visibilidad (acceso) de los elementos.

Un paquete es un contenedor lógico de entidades que tienen una funcionalidad relacionada.

Es un concepto similar al de una carpeta del FS, que agrupa archivos relacionados.

El concepto de comunicación entre objetos es válido dentro de un mismo programa, a su vez dentro de un mismo dispositivo.

Cuando se trata de comunicación a través de la red, los objetos de una aplicación que deseen comunicarse con objetos remotos de otra aplicación, deben hacerlo usando los mecanismos y medios estándares que ofrece la plataforma. Esto lleva a que (salvo algunos casos especiales) los objetos existentes en un extremo, deban ser transformados antes de ser trasferidos y luego reconstruidos en el otro extremo.

Si bien existen diversas formas de comunicar 2 (o más) dispositivos, el modelo de interés aquí es el denominado Cliente/Servidor. Donde uno de los nodos (el Servidor) es capaz de proveer uno o más servicios que son consumidos por otro nodo (el Cliente). Cabe aclarar que en este modelo o arquitectura de red, puede haber varios Clientes consumidores concurrentes, incluso alguno puede encontrarse en el mismo equipo donde corre el servidor.

Requerimientos:

Esta actividad tiene la finalidad de diseñar 3 paquetes, destinados a diferentes nodos de una red, a partir del código fuente C++ entregado.

Este código usa una librería externa, que permite operar usando XML y RPC combinados, para intercambiar información entre dispositivos de esa red.

Cada uno de los paquetes debe contener al menos 1 clase y un módulo de lanzamiento separado. Los paquetes previstos corresponden 1 al lado servidor, y 2 a diferentes clientes.

Restricciones al diseño e implementación:

- ✓ Las librerías de terceros usadas deben encontrarse dentro de un directorio denominado lib (o library)
- ✓ Los archivos de cabecera propios deben encontrarse en un directorio denominado inc (o include)
- ✓ Cada paquete debe tener su propio directorio de código fuente
- ✓ El diseño OO debe separar la capa de presentación/control de la de modelo.
- ✓ Cambie los controles que usan if y mensajes directos a un modelo de gestión de errores mediante clases de excepción personalizadas, en los escenarios básicos de operación previstos.
- ✓ Incorporar guardas a las clases y espacios de nombre apropiados, asociados a cada package.

Descripción complementaria

El ejemplo provisto responde a servidor RPC capaz de proveer servicios, a uno o más clientes capaces de consumir dichos servicios.

El Servidor posee las siguientes características:

- ✓ Provee de diferentes servicios principales (prueba trivial, respuesta simple y cálculo) y auxiliares (de ayuda para guía o con sintaxis esperada, ya sea nativos de la librería o extendidos a partir de ella).

- ✓ Pone los valores a disposición de los clientes mediante XML-RPC o JSON-RPC, en una dirección IP y un puerto informados.
- ✓ Aplicación básica de línea de comandos

Existen 2 (dos) aplicaciones para el lado Cliente, con las siguientes características:

- ✓ Son aplicaciones de línea de comandos para prueba del servicio,
- ✓ Una para peticiones individuales y directas. Otra con un loop de peticiones preestablecidas.
- ✓ Despliegan algún tipo de ayuda mínima al operador, en caso que se equivoque.
- ✓ Producen una salida en pantalla luego de cada petición realizada al servidor.

Procedimiento general:

1) A partir del ejemplo provisto, rediseñe bajo principios de OO, cumpliendo las siguientes restricciones mencionadas anteriormente.

2) Generar los módulos que correspondan en lenguaje C++.

3) Realizar un informe similar a los anteriores.

4) Agregar una captura que muestre la distribución de los archivos en el árbol de directorios resultante

5) Agregue un gráfico con el diagrama de clases post desarrollo (mediante análisis o ingeniería inversa) donde se observen los packages del lado cliente y el lado servidor.

6) Preparar un archivo comprimido, similar a lo pautado en entregas anteriores.

7) Realizar la entrega del informe y del archivo comprimido con la implementación.

Recursos complementarios (disponibles en el aula virtual)**:**

- Apuntes de clase.
- Código de base con librerías complementarias (base_xml_rpc)
- Referencias y ejemplos provistos con documentación sobre protocolos, comunicaciones en redes y librerías de terceros.